SIMATS ENGINEERING

ASSESSMENT TOOL 1: AI Model Design Exercise

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA0101**

COURSE OUTCOME COVERED

CO4: *Develop intelligent software agents using suitable agent architectures and learning techniques.*(BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 1

Annexure A

AI Model Design Exercise

Code of Conduct:

I Shaik Mubarak(192425194)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Dataset Selection and Data Understanding	10%	
3. Data Preprocessing and Feature Engineering	10%	
4. AI Technique Selection and Justification	10%	
5. Model Design / Architecture	10%	
6. Algorithm / Pseudocode Development	5%	
7. Model Implementation and GitHub Repository	15%	
8. Experimental Design and Results	10%	
9. Performance Evaluation and Metrics	10%	
10. Result Analysis and Interpretation	5%	
11. Limitations and Future Enhancements	5%	
12. Documentation and Technical Presentation	10%	
Total	100%	

Signature of the Faculty Total Marks Awarded

ANSWER ANY ONE QUESTION

S.No.	Scenario / Problem Statement	AI Model /Technique to be Developed	Expected Development Outcome
1	**Smart Loan Approval:** A bank wants to automatically determine whether a customer is eligible for a loan using income, credit score, employment status, loan amount, and repayment history.	Decision Tree	Design and implement a decision-tree-based AI model and determine the most suitable attribute-selection measure such as Information Gain, Gain Ratio, or Gini Index.
2	**Student Performance Prediction:** A university wants to predict whether a student is likely to pass or fail a course using attendance, internal marks, assignment performance, study hours, and previous academic performance.	Inductive Learning	Construct a predictive model from training examples and derive generalized decision rules from the learned patterns.
3	**Disease Risk Prediction:** A healthcare system needs to classify patients into low-risk and high-risk categories based on clinical measurements.	Statistical Learning	Design a statistical learning model, train it using patient data, and evaluate its generalization ability on unseen data.
4	**Image Recognition System:** An organization wants to automatically classify images into categories such as vehicles, animals, and buildings.	Neural Network	Design and implement a neural-network classifier by specifying the network architecture, activation functions, loss function, optimizer, and training procedure.
5	**Handwritten Character Recognition:** A system must recognize handwritten characters from scanned images.	Multilayer Neural Network	Develop a multilayer neural network and demonstrate how backpropagation updates weights to minimize classification error.

6	**Non-Linear Classification:** A dataset contains two classes that cannot be separated using a straight-line decision boundary.	SVM with Kernel Method	Design an SVM-based classifier using an appropriate kernel function and justify the selected kernel based on the characteristics of the dataset.
7	**Intelligent Recommendation Agent:** An online platform wants an agent that learns which recommendations to provide based on user responses.	Reinforcement Learning	Design an RL model by defining the states, actions, rewards, policy, and learning strategy, and demonstrate how the agent learns from user feedback.
8	**Autonomous Game Agent:** An AI agent must learn to play a game where successful actions receive positive rewards and unsuccessful actions receive penalties.	Reinforcement Learning	Develop an RL-based game agent and demonstrate how its policy and performance improve through repeated interaction with the environment.
9	**Explainable Diagnosis System:** A medical AI system predicts a disease but must also provide an explanation for its prediction.	Explanation-Based Learning	Design an explanation-based learning approach using domain knowledge and training examples to generate an understandable justification for the prediction.
10	**AI Model Selection Challenge:** A company provides a dataset for predicting customer churn.	Comparative AI Model Design	Design and compare at least two learning approaches from decision trees, statistical learning, neural networks, or kernel methods. Select the best model based on accuracy, interpretability, computational requirements, and generalization ability.

SMART LOAN APPROVAL USING DECISION TREE CLASSIFICATION

1. Problem Statement

Banks receive a large number of loan applications from customers. Traditionally, bank employees examine various factors such as the applicant's income, credit history, employment status, loan amount, and repayment capability before approving or rejecting a loan. This manual process can be time-consuming and may lead to inconsistent decisions.

The objective of this project is to develop an AI-based Smart Loan Approval System that automatically predicts whether a customer's loan application should be Approved or Rejected.

Input Data

The model uses applicant-related information such as:

- Applicant income
- Co-applicant income
- Credit history
- Employment status
- Education
- Marital status
- Number of dependents
- Loan amount
- Loan term
- Property area

Expected Output

The system predicts one of the following classes:

- Loan Approved
- Loan Not Approved

AI Task

This is a supervised machine learning binary classification problem. A Decision Tree classifier is used to learn patterns from historical loan application data.

2. Objectives

The major objectives of the proposed system are:

1. To develop an automated system for predicting loan approval.
2. To analyze the relationship between applicant information and loan approval status.
3. To preprocess the loan dataset by handling missing values and categorical attributes.
4. To develop a Decision Tree-based classification model.
5. To investigate suitable attribute-selection measures such as Information Gain, Gain Ratio, and Gini Index.
6. To evaluate the performance of the developed model using appropriate classification metrics.
7. To identify the most important factors affecting loan approval.
8. To reduce the time required for preliminary loan eligibility decisions.

3. Dataset Description

For this project, a Loan Approval Prediction Dataset can be used.

The dataset contains historical information about loan applicants and the final status of their loan applications.

Important Dataset Attributes

Attribute	Description	Data Type
Loan_ID	Unique identification number	Categorical
Gender	Gender of the applicant	Categorical

Married	Marital status	Categorical
Dependents	Number of dependents	Categorical
Education	Graduate/Not Graduate	Categorical
Self_Employed	Employment status	Categorical
ApplicantIncome	Income of the applicant	Numerical
CoapplicantIncome	Income of co-applicant	Numerical
LoanAmount	Requested loan amount	Numerical
Loan_Amount_Term	Loan repayment period	Numerical
Credit_History	Previous credit information	Numerical
Property_Area	Urban/Rural/Semiurban	Categorical
Loan_Status	Loan approval status	Target Variable

Target Variable

Loan_Status

- **Y** → Loan Approved
- **N** → Loan Not Approved

AI Learning Type

Supervised Learning – Binary Classification

The model learns from previously labeled loan applications and predicts the status of new applications.

4. Data Preprocessing

Raw datasets may contain missing, inconsistent, or categorical values. Therefore, preprocessing is required before training the AI model.

Step 1: Data Cleaning

- Remove duplicate records if present.
- Check the data types of all attributes.
- Remove unnecessary columns such as Loan_ID.

Step 2: Missing Value Handling

Missing categorical values can be replaced using the **mode**.

Example:

- Gender → Most frequent gender
- Married → Most frequent marital status
- Self_Employed → Most frequent value

Missing numerical values can be replaced using the **median**.

Example:

- LoanAmount → Median loan amount
- Loan_Amount_Term → Median loan term

Step 3: Encoding Categorical Data

Machine learning models require numerical input. Therefore, categorical attributes are converted into numerical form.

Examples:

Original Value	**Encoded Value**
Male	1
Female	0
Yes	1
No	0
Graduate	1
Not Graduate	0

For attributes containing more than two categories, One-Hot Encoding can be used.

Step 4: Feature Selection
Important features include:

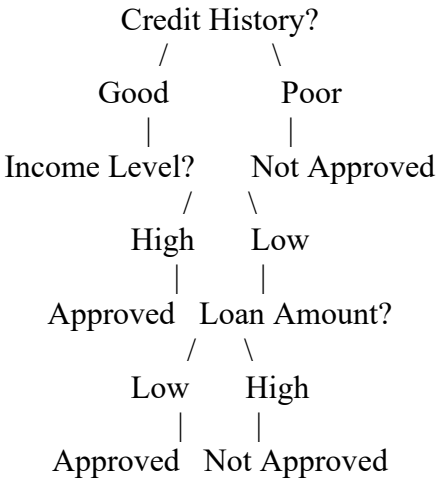
- ApplicantIncome
- CoapplicantIncome
- LoanAmount
- Credit_History
- Education
- Self_Employed
- Property_Area

Step 5: Train-Test Split
The dataset is divided into:

- 80% Training Data
- 20% Testing Data

The training data is used to build the Decision Tree, while the testing data is used to evaluate its performance.

5. Selected AI Technique and Justification
Selected Technique: Decision Tree Classifier
A Decision Tree is selected because the problem requires classification into two categories: Approved or Not Approved.
A Decision Tree works by dividing the dataset into smaller groups based on the most important attributes.
Example Decision Process



Why Decision Tree?

1. Easy to understand.
2. Easy to visualize.
3. Works with numerical and categorical data.
4. Provides interpretable decision rules.
5. Requires relatively less complex preprocessing.
6. Can identify important features.
7. Suitable for loan approval classification.

Attribute-Selection Measures
A. Information Gain
Information Gain measures the reduction in uncertainty after splitting the data based on an attribute.
The attribute with the highest Information Gain is selected for splitting.

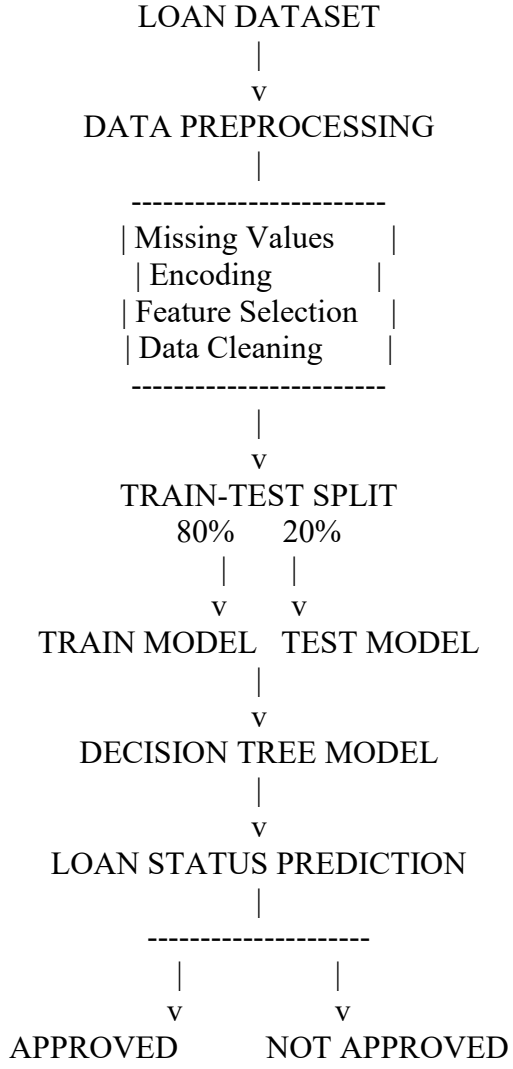
B. Gain Ratio
Gain Ratio improves Information Gain by reducing bias toward attributes with many possible values.
C. Gini Index
The Gini Index measures the impurity of a dataset.
A lower Gini value indicates a better split.
Selected Measure
For the implementation, Gini Index can be selected because:

- It is computationally efficient.
- It is the default criterion in many Decision Tree implementations.
- It works effectively for classification problems.

Entropy can also be tested and compared with Gini Index.

---

6. Model Design / Architecture
The overall architecture of the proposed system is:



Model Input
The input consists of applicant features:
Model Output
Important Hyperparameters

- criterion = "gini" or "entropy"

- max_depth
- min_samples_split
- min_samples_leaf
- random_state

7. Algorithm / Pseudocode
Decision Tree Loan Approval Algorithm
START

1. Load the loan approval dataset.

2. Display the dataset information.

3. Remove unnecessary columns.

4. Check for missing values.

5. Replace missing categorical values using mode.

6. Replace missing numerical values using median.

7. Convert categorical values into numerical values.

8. Separate input features X and target variable Y.

9. Divide the dataset into training and testing sets.

10. Create a Decision Tree classifier.

11. Train the classifier using training data.

12. Predict loan status using testing data.

13. Compare predicted values with actual values.

14. Calculate:
    Accuracy
    Precision
    Recall
    F1-score

15. Generate the confusion matrix.

16. Compare Decision Tree criteria if required.

17. Select the best-performing model.

18. Predict the loan status for new customer data.

STOP

8. Implementation

Programming Language

Python

Tools and Libraries

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Important Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Main Implementation Steps

1. Import required libraries.
2. Load the CSV dataset.
3. Examine dataset structure.
4. Handle missing values.
5. Encode categorical attributes.
6. Split data into training and testing sets.
7. Create a Decision Tree model.
8. Train the model.
9. Make predictions.
10. Evaluate model performance.
11. Display confusion matrix.
12. Visualize the Decision Tree.

9. Experimental Results

The experimental results should be presented after executing the Python program.

Example Results Table

Model	Criterion	Accuracy	Precision	Recall	F1-Score
Decision Tree 1	Gini Index	XX%	XX%	XX%	XX%
Decision Tree 2	Entropy / Information Gain	XX%	XX%	XX%	XX%

Important: Replace the XX% values with the actual values obtained when you run your code.

Sample Prediction Table

Applicant	Actual Status	Predicted Status
Customer 1	Approved	Approved
Customer 2	Not Approved	Not Approved
Customer 3	Approved	Approved
Customer 4	Not Approved	Approved

Confusion Matrix

	Predicted Not	Approved
Actual Not	TN	FP
Actual Approved	FN	TP

The confusion matrix helps analyze correct and incorrect loan approval predictions.

10. Performance Evaluation

The Decision Tree model can be evaluated using the following metrics.

Accuracy

Accuracy represents the percentage of correct predictions.

Precision

Precision measures how many predicted approved loans were actually approved.

Recall

Recall measures how many actual approved loans were correctly identified.

F1-Score

The F1-score balances Precision and Recall.

11. Result Analysis

The Decision Tree model learns the relationship between customer characteristics and loan approval status.

Important observations may include:

- Credit history may have a strong influence on loan approval.
- Applicants with a good credit history are more likely to receive approval.
- Income and loan amount affect the customer's repayment capability.
- Employment status may provide information about financial stability.
- The Decision Tree provides understandable rules for decision-making.

The Gini Index-based and Entropy-based Decision Trees can be compared using accuracy and other evaluation metrics.

The model with better test performance and lower complexity should be selected as the final model.

Strengths

- Easy to interpret.
- Fast prediction.
- Produces understandable decision rules.
- Identifies important attributes.

Weaknesses

- Can overfit if the tree becomes very deep.
- Performance depends on data quality.
- Biased or imbalanced data can affect predictions.

12. Limitations

The proposed system has several limitations:

1. The model depends heavily on the quality of historical data.
2. Missing or incorrect data can reduce prediction accuracy.
3. A Decision Tree can overfit the training dataset.
4. The dataset may not contain all real-world factors considered by banks.
5. Economic conditions may change over time.
6. A simple model cannot completely replace professional financial assessment.
7. Biased historical decisions can produce biased predictions.
8. The model should not be used as the only basis for final loan decisions.

13. Future Enhancements

The system can be improved in the future by:

1. Using a larger and more recent dataset.
2. Adding more financial attributes.
3. Performing advanced feature engineering.
4. Applying hyperparameter tuning.
5. Comparing Decision Tree with Random Forest and Gradient Boosting.
6. Handling class imbalance using appropriate techniques.
7. Using Explainable AI techniques to explain individual predictions.
8. Deploying the model as a web application.
9. Developing a real-time loan eligibility prediction system.
10. Adding fairness and bias detection mechanisms.

14. Conclusion

This project presented a **Smart Loan Approval System using a Decision Tree classifier**. The purpose of the system is to automatically predict whether a customer's loan application is likely to be approved or rejected based on information such as income, employment status, loan amount, credit history, and other relevant attributes.The dataset is preprocessed by handling missing values, encoding categorical variables, selecting relevant features, and dividing the data into training and testing sets. A Decision Tree model is then trained to learn the patterns associated with loan approval.

Different attribute-selection measures, including Information Gain, Gain Ratio, and Gini Index, are considered. Gini Index is suitable for efficient classification, while Information Gain and Gain Ratio provide alternative methods for selecting the most informative attributes. The final model can be evaluated using accuracy, precision, recall, F1-score, and a confusion matrix. The proposed system demonstrates how AI and machine learning can support faster and more consistent preliminary loan eligibility assessment. However, the model should be used as a decision-support tool rather than as the sole authority for financial decisions.

15. References

For your final report, include references in a format like this:

1. Loan Approval Prediction Dataset, dataset source used for the project.
2. Scikit-learn Documentation, Decision Tree Classifier.
3. T. M. Mitchell, *Machine Learning*, McGraw-Hill.
4. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer.
5. J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*.
6. Python Documentation.
7. Pandas Documentation.
8. Scikit-learn Machine Learning Library Documentation.

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Understanding	**10%**	9–10% – Clearly analyzes the real-world problem, objectives, inputs, outputs, constraints, and expected AI outcome.	7–8% – Understands the problem and objectives with minor omissions.	5–6% – Shows partial understanding with some important elements missing.	0–4% – Problem is poorly understood or incorrectly defined.

2. Dataset Selection and Understanding	**10%**	9–10% – Selects a highly relevant dataset and clearly explains features, target, source, size, and characteristics.	7–8% – Appropriate dataset with adequate description and minor omissions.	5–6% – Dataset is usable but description or relevance is limited.	0–4% – Dataset is inappropriate, poorly described, or missing.
3. Data Preprocessing and Feature Engineering	**10%**	9–10% – Performs appropriate cleaning, transformation, encoding, scaling, feature selection, and data splitting with clear justification.	7–8% – Performs most required preprocessing correctly with minor issues.	5–6% – Basic preprocessing is performed but important steps are missing.	0–4% – Preprocessing is inadequate, incorrect, or absent.
4. AI Technique Selection and Justification	**10%**	9–10% – Selects the most appropriate AI technique and provides strong technical justification based on the problem and dataset.	7–8% – Appropriate technique selected with reasonable justification.	5–6% – Technique is acceptable but justification is limited.	0–4% – Inappropriate technique or no meaningful justification.
5. Model Design / Architecture	**10%**	9–10% – Develops a clear, technically sound model architecture with appropriate components, parameters, and workflow.	7–8% – Model is well designed with minor omissions.	5–6% – Basic model design is presented but lacks technical depth.	0–4% – Model design is unclear, incomplete, or inappropriate.
6. Algorithm / Pseudocode	**5%**	5% – Algorithm/pseudocode is complete, logically correct, structured, and clearly represents the model development process.	4% – Mostly correct with minor omissions.	2–3% – Basic algorithm provided but contains gaps.	0–1% – Algorithm is missing or substantially incorrect.

7. Implementation and GitHub Repository	**15%**	13–15% – Fully functional implementation with clean, organized, reproducible code and a complete GitHub repository with README and supporting files.	10–12% – Functional implementation and well-organized repository with minor issues.	7–9% – Partially functional implementation or incomplete repository documentation.	0–6% – Implementation is incomplete/non-functional or GitHub repository is missing.
8. Experimental Design and Results	**10%**	9–10% – Conducts meaningful experiments with appropriate test cases and clearly presents results using tables, graphs, or visualizations.	7–8% – Appropriate experiments with clearly presented results.	5–6% – Limited experiments or basic result presentation.	0–4% – Insufficient, incorrect, or missing experimental results.
9. Performance Evaluation	**10%**	9–10% – Uses appropriate evaluation metrics and provides comprehensive quantitative analysis of model performance.	7–8% – Uses suitable metrics and provides adequate evaluation.	5–6% – Basic evaluation with limited metrics or analysis.	0–4% – Inappropriate metrics or performance evaluation is missing.
10. Result Analysis and Interpretation	**5%**	5% – Provides insightful analysis of results, identifies patterns, errors, strengths, weaknesses, and explains model behavior.	4% – Provides good interpretation with minor gaps.	2–3% – Basic interpretation with limited analysis.	0–1% – Results are not meaningfully analyzed.
11. Limitations and Future Enhancements	**5%**	5% – Clearly identifies realistic limitations and proposes technically relevant and feasible improvements.	4% – Identifies relevant limitations and reasonable improvements.	2–3% – Provides basic limitations and future suggestions.	0–1% – Missing, unrealistic, or poorly explained.

12. Documentation and Technical Presentation	**10%**	9–10% – Report is comprehensive, well organized, technically accurate, professionally presented, and properly referenced.	7–8% – Well documented with minor presentation or referencing issues.	5–6% – Adequate documentation with several gaps.	0–4% – Poorly documented, unclear, or incomplete report.
Total	100%				